CSE 4345: Big Data Analytics

Week 1 Lecture Notes

Digital Data Landscape, Introduction to Big Data, and the 3Vs Framework

Department of Computer Science & Engineering | Premier University, Chittagong

CLO: CLO1 — Understand big data characteristics **PLO:** PLO(a)

After studying these notes you will be able to:

- Define Big Data using the 3Vs (and 5Vs) framework with real-world examples
- Distinguish between structured, semi-structured, and unstructured data
- Explain why traditional relational databases cannot handle Big Data workloads
- Differentiate between vertical and horizontal scaling
- State and apply the CAP theorem to distributed database design
- Compare batch processing and stream processing and identify suitable use cases for each

1. The Data Revolution: Why Now?

Every minute of 2024, the world generates staggering volumes of data: 500 hours of video are uploaded to YouTube, 695,000 Instagram Stories are shared, 16 million text messages are sent, and over 6,000 tweets are posted. This scale is not just large — it is qualitatively different from anything that existed a generation ago.

1.1 Historical Perspective

Data has always existed, but the **rate of generation and the cost of storage** have changed beyond recognition:

Era	Data Sources	Scale	Dominant Tool
1960s–80s	Census, financial records	Megabytes	Mainframes
1990s–2000s	Web clicks, e-commerce logs	Gigabytes – Terabytes	RDBMS (Oracle, MySQL)
2010s	Social media, smartphones, IoT	Petabytes – Exabytes	Hadoop, NoSQL
2020s	AI training data, genomics, autonomous vehicles	Zettabytes	Data lakes, lakehouse, streaming

Key Insight By 2025, the IDC *Global DataSphere* report forecasts the world's total stored data will reach **175 zettabytes** (175×10^{21} bytes). If you stored this on DVDs, the stack would circle Earth 222 times.

1.2 Why Organizations Care

The McKinsey Global Institute (2011) report *"Big Data: The Next Frontier for Innovation, Competition, and Productivity"* (Manyika et al.) quantified the business case:

- Retailers applying big data analytics could increase operating margins by **more than 60%**
- The US healthcare system could unlock **over $300 billion** in annual value through clinical analytics
- Big data is now a **factor of production** — alongside capital, land, and labor — that creates economic surplus

For Bangladesh specifically: the fintech sector (bKash, Nagad) processes hundreds of millions of transactions daily; DGHS manages health records for 170 million people; the RMG sector tracks logistics for thousands of factories. Each of these is a big data problem demanding techniques beyond what a standard relational database can provide.

2. Defining Big Data: The 3Vs Framework

The term "Big Data" is often used loosely. Its most widely accepted technical definition comes from a 2001 research note by **Doug Laney** at the META Group (later Gartner):

Definition (Laney, 2001)
Big Data is characterized by three dimensions:
Volume (scale of data), **Velocity** (speed of generation and processing), and **Variety** (diversity of data types). Data is "big" when its Volume, Velocity, or Variety exceeds the capacity of conventional database tools to capture, manage, and process it within a tolerable elapsed time.

These three dimensions are not independent — a system often faces all three simultaneously.

2.1 Volume: Scale of Data

Volume refers to the **sheer quantity** of data. The challenge is not just storage cost, but the inability of single-machine algorithms (which assume all data fits in RAM or even on one disk) to process it.

Organization	Data Volume	System Used
Facebook (Meta)	500 TB ingested per day; 300 PB total	Apache Hive on Hadoop
Google	20 PB processed per day by MapReduce	GFS + Bigtable
CERN (LHC)	15 PB generated per year from particle collisions	Custom distributed stores
bKash (Bangladesh)	>500 million transactions per year	Cloud + Hadoop stack
NID Database (Bangladesh)	∼110 million citizen records	Government data center

Unit progression: 1 KB → 1 MB → 1 GB → 1 TB → 1 PB → 1 EB → 1 ZB. Each step is a factor of 1,024. A single genome sequence is ∼200 GB; sequencing all humans on Earth would produce ∼1.5 ZB.

2.2 Velocity: Speed of Generation and Processing

Velocity refers to **how fast data arrives** and, crucially, **how quickly it must be processed** to be useful.

Source	Generation Rate	Processing Requirement
Twitter / X	6,000 tweets/second	Near real-time trend detection
Credit card transactions	45,000 tx/second (Visa peak)	Fraud detection in <100 ms
IoT sensors (factory floor)	Millions of readings/second	Predictive maintenance
Pathao GPS pings	Every 3 seconds per active driver	Real-time routing
Stock exchange ticks	>1 million/second	Algorithmic trading in microseconds

Key Insight Velocity is why batch processing (e.g., running a nightly Hadoop job) is insufficient for some use cases. A fraud detection system that reports suspicious transactions the *next morning* is useless — the money is already gone.

2.3 Variety: Diversity of Data Types

Variety refers to the **diversity of formats and structures** in which data arrives.

Type	Characteristics	Examples
Structured	Fixed schema, tabular, easy to query	SQL databases, CSV files, spreadsheets
Semi-structured	Self-describing schema, flexible	JSON, XML, log files, email headers
Unstructured	No predefined format, hard to parse	Images, video, audio, free-text documents, MRI scans

A key fact worth memorizing: **structured data represents only ∼20% of all digital data**. The remaining ∼80% is unstructured — images, videos, documents, social media posts. Traditional RDBMS tools were built for structured data only, which is why big data ecosystems (Hadoop, Spark) extend to semi-structured and unstructured formats.

2.4 The 5Vs Extension

Later scholars and Gartner analysts extended Laney's 3Vs to acknowledge two additional dimensions:

V	Meaning	Example Challenge
Volume	Quantity of data	500 TB/day at Facebook
Velocity	Speed of generation and required processing	Fraud detection in $<$100 ms
Variety	Diversity of formats and structures	Mixing JSON logs with MRI images
Veracity	Uncertainty and quality of data	Sensor dropout, OCR errors, bots
Value	Business utility extracted from data	Revenue from predictive analytics

Gartner Definition (Beyer & Laney, 2012)
"Big data is high-volume, high-velocity and/or high-variety information assets that demand cost-effective, innovative forms of information processing that enable enhanced insight, decision making, and process automation."

Common Misconception: "Big Data" does not simply mean "large dataset." A 10 TB single flat file with no velocity or variety requirements can be handled by a well-tuned PostgreSQL instance. Bigness is relative to your processing capacity, not an absolute number.

3. Types of Digital Data

Understanding data types is foundational because each type requires **different storage systems, query engines, and analytics approaches**.

3.1 Structured Data

Structured data follows a **predefined schema**: every record has the same fields, with the same data types, in the same order. It is the native format of relational database management systems (RDBMS).

Characteristics:

- Schema is enforced at write time (schema-on-write)
- Easily queried with SQL: `SELECT`, `JOIN`, `GROUP BY`
- Optimized for OLTP (Online Transaction Processing): fast inserts, point queries
- Represents approximately **20%** of all digital data

Examples: bank transaction tables (account number, amount, timestamp, branch code), hospital patient records (ID, name, DOB, diagnosis code), bKash wallet balance tables.

3.2 Semi-Structured Data

Semi-structured data has **some organizational structure** but does not conform to a rigid tabular schema. It is self-describing through tags, keys, or markers.

Characteristics:

- Schema is flexible and can vary between records (nested, repeated fields)
- No strict schema enforcement at ingestion
- Queryable via path expressions (JSON path, XPath) or schema-on-read tools (Hive, Spark SQL)

Examples:

- **JSON**: `{"user": "Alice", "amount": 1500, "tags": ["food","online"]}` — common in web APIs
- **XML**: used in HL7 FHIR health record exchange between hospitals
- **Apache web logs**: `192.168.1.1 - - [12/Jun/2024:10:34:01] "GET /api/balance HTTP/1.1" 200 345`
- **Email headers**: structured fields (From, To, Subject) + unstructured body

3.3 Unstructured Data

Unstructured data has **no predefined format or schema**. It is the hardest to store, query, and analyse — and yet it is the most abundant form of data (∼80% of all digital data).

Characteristics:

- Cannot be stored meaningfully in relational tables without heavy preprocessing
- Requires machine learning (computer vision, NLP, speech recognition) to extract meaning
- Often stored as files in distributed file systems (HDFS, S3) and object stores

Examples: CCTV video footage, medical imaging (X-ray, MRI, CT scan), satellite imagery (used by Bangladesh Meteorological Department), WhatsApp voice messages, court documents, social media posts in Bangla.

Example: Healthcare Domain

A hospital generates all three types simultaneously. The *patient demographics table* (name, DOB, NID, blood type) is structured. The *doctor's notes* in the Electronic Health Record are unstructured free text. The *HL7 FHIR discharge summary* transmitted to another hospital is semi-structured XML. Big data platforms must ingest and correlate all three.

4. Why Traditional Databases Cannot Handle Big Data

Traditional RDBMS (MySQL, Oracle, PostgreSQL) were designed for a world where:

1. Data fits on a single server (or a small cluster with shared storage)
2. Queries are transactional and require ACID guarantees
3. The schema is known in advance and changes rarely

When data exceeds these assumptions, three critical breakdowns occur:

Limitation	RDBMS Behavior	Big Data Consequence
Single-node capacity	Maximum data = one server's disk (a few TB)	Cannot store 500 TB; requires distributed storage
Query latency	Full-table scan on 1 TB = hours	Analytics teams wait unacceptably long
Schema rigidity	ALTER TABLE on 1 TB table = downtime	Cannot evolve schema as requirements change
Variety blindness	No support for images, JSON trees, streams	Cannot analyse 80% of enterprise data
Concurrency under load	Write locks slow throughput	Cannot handle 45,000 tx/second ingestion

4.1 Vertical vs. Horizontal Scaling

The classical database solution to capacity problems is **vertical scaling** (scale up): buy a bigger, faster server with more RAM, faster CPU, and larger disks.

	Vertical Scaling (Scale Up)	Horizontal Scaling (Scale Out)
Approach	Add more RAM/CPU/disk to one machine	Add more machines to a cluster
Cost	Exponential — high-end servers cost 10–100× more for 2× capacity	Linear — commodity servers (1,000 USD each)
Fault tolerance	Single point of failure	Redundancy built across nodes
Upper limit	Physical hardware limit (∼few TB RAM, few PB disk)	Effectively unlimited (Google: millions of nodes)
RDBMS fit	Natural fit; RDBMS was designed for this	Requires redesign of data and query model
Big data approach	Not used	**Core design principle of Hadoop, Spark, Cassandra**

Key Insight Hadoop's entire design philosophy is "bring computation to data, not data to computation." Rather than moving petabytes to a central processing unit, Hadoop sends the processing code to each node where the data already lives. This **data locality** principle is the reason horizontal scaling works for big data analytics.

5. The CAP Theorem

As soon as data is distributed across multiple machines (horizontal scaling), a fundamental question arises: *what guarantees can the system make when the network between nodes fails?* Eric Brewer formally answered this in 2000.

CAP Theorem (Brewer, 2000; formalized by Gilbert & Lynch, 2002)
A distributed data store can simultaneously guarantee **at most two** of the following three properties:

- **Consistency (C):** Every read receives the most recent write (or an error) — all nodes see the same data at the same time
- **Availability (A):** Every request receives a (non-error) response, though it may not contain the most recent data
- **Partition Tolerance (P):** The system continues to operate even when network messages between nodes are lost or delayed (a network partition)

5.1 Understanding the Three Properties

Consistency means that if you write a value to node A, and immediately read it from node B (a different server), you get the written value — not an old cached copy. This is what you expect from a traditional bank transfer: if you deposit $100, your balance on any ATM worldwide should reflect that immediately.

Availability means every request gets a response. The system never rejects a query with "I cannot answer right now." Even if some nodes are down, the remaining nodes serve requests.

Partition Tolerance means the system keeps working even when network communication between nodes breaks. In a real distributed system — spanning data centers, countries, or even just two racks in one building — network partitions are **not hypothetical**. They happen regularly due to cable cuts, switch failures, and routing errors.

5.2 The Critical Implication

Since network partitions **will occur** in any real distributed system, Partition Tolerance (P) is non-negotiable for large-scale systems. This reduces the real choice to:

- **CP systems** (Consistency + Partition Tolerance): When a partition occurs, reject some requests to ensure the data that *is* returned is always correct. **Examples:** HBase, ZooKeeper, MongoDB (default config), Google Spanner
- **AP systems** (Availability + Partition Tolerance): When a partition occurs, continue serving requests even if responses may return slightly stale data. **Examples:** Cassandra, DynamoDB, CouchDB

System	CAP Choice	Reason / Trade-off
Traditional RDBMS (MySQL)	CA	Runs on one node; partitions don't apply
HBase	CP	Prefers to reject requests than serve stale data (banking)
Apache Cassandra	AP	Eventual consistency; prefers uptime (social feed, IoT)
Amazon DynamoDB	AP (tunable)	Defaults to eventual consistency; strong consistency optional
Google Spanner	CP (global)	Uses GPS + atomic clocks to achieve strong consistency at scale
ZooKeeper	CP	Coordination service: consistency is paramount

Example: bKash Mobile Banking

bKash processes financial transfers. It uses a **CP-leaning** design: if the system cannot guarantee that a transfer is atomically applied across all replicas, it *refuses* the transaction rather than risking a double-spend or balance discrepancy. Availability is sacrificed briefly (you may get a "try again" error) to protect Consistency. Contrast this with a social media feed (like Facebook's News Feed), which uses an **AP design**: it is acceptable to see a post 2 seconds late; it is not acceptable for the feed to go down.

Common Misconception: The CAP theorem does not mean "choose two, ignore one completely." Modern systems (like Cassandra with tunable consistency, or Google Spanner) are designed to *minimize* the cases where the trade-off bites, using techniques like quorum reads/writes and synchronized clocks. Think of CAP as describing behavior *during a partition*, not normal operation.

6. Batch Processing vs. Stream Processing

Once data is stored in a distributed system, there are two fundamentally different paradigms for processing it:

Batch Processing

Processing of data **collected over a period of time** as a single bounded job. The system reads all input, applies a computation, and produces output. The input dataset is complete and static during processing.
Canonical system: Apache Hadoop MapReduce.

Stream Processing
Processing of data **as it arrives**, record by record or in micro-batches, over an unbounded (potentially infinite) data stream. The system produces output continuously, often within milliseconds of data arrival.
Canonical systems: Apache Kafka, Apache Flink, Spark Structured Streaming.

6.1 Side-by-Side Comparison

Property	Batch Processing	Stream Processing
Data bounds	Bounded (finite dataset)	Unbounded (infinite stream)
Latency	Minutes to hours	Milliseconds to seconds
Throughput	Very high (optimized for bulk)	Lower per unit time
Complexity	Lower (no state management)	Higher (windowing, watermarks, state)
Fault tolerance	Re-run from input file	Checkpointing, replay from Kafka
Output	Written when job completes	Continuous (updated results)
Typical use case	Nightly ETL, monthly reports, model training	Fraud detection, live dashboards, alerting
Examples	Hadoop MR, Hive, Spark batch	Kafka Streams, Flink, Spark Streaming

6.2 Choosing the Right Paradigm

The decision between batch and stream processing is driven by **how quickly the result must be available after the data arrives**:

- **Use batch when:** results are needed hours or days after data collection (monthly sales reports, end-of-day bank reconciliation, weekly model retraining, DGHS annual disease statistics)
- **Use stream when:** results are needed within seconds of data arrival (bKash fraud alert, Pathao surge pricing when 100 drivers go offline simultaneously, hospital patient monitoring alarms, stock trading signals)
- **Use both (Lambda / Kappa architecture):** many production systems combine batch for historical accuracy and stream for low-latency approximations (covered in Week 8)

Example: Ride-Sharing: Pathao in Bangladesh

Pathao collects GPS pings from drivers every 3 seconds. **Stream processing** is used to update the live map of available drivers in real time (latency requirement: <1 second). **Batch processing** is used to compute weekly driver earnings reports, trip analytics, and demand heatmaps by neighbourhood (latency requirement: hours to days, but covering millions of historical records). Both pipelines run in parallel on the same underlying data.

7. Summary and Key Takeaways

Concept	What to Remember
3Vs (Laney, 2001)	Volume, Velocity, Variety — the canonical definition of Big Data; extended to 5Vs with Veracity and Value
Structured data	∼20% of all data; fixed schema, queryable with SQL; RDBMS territory
Unstructured data	∼80% of all data; images, video, free text; requires ML to process
Vertical scaling	Add more to one machine; expensive; has a hard limit
Horizontal scaling	Add more machines; linear cost; the core principle of Hadoop/Spark
CAP theorem	Distributed systems can guarantee only 2 of C, A, P simultaneously; since P is required, real choice is CP vs. AP
CP systems	Consistency + Partition Tolerance; reject requests during partition (e.g., HBase, ZooKeeper)
AP systems	Availability + Partition Tolerance; return possibly stale data during partition (e.g., Cassandra, DynamoDB)
Batch processing	Bounded input, high throughput, minutes-to-hours latency; nightly ETL, reports
Stream processing	Unbounded input, low latency (ms–s); fraud detection, live dashboards

Key Insight The three Vs of Big Data (Volume, Velocity, Variety) define the *problem*. Horizontal scaling, the CAP theorem, and batch/stream architectures define the *engineering responses* to that problem. Everything in this course (Hadoop, Spark, Hive, Kafka) is an implementation of those responses.

8. Self-Assessment

Multiple Choice Questions

Instructions: Select the single best answer.

Q1. Who first formally introduced the 3Vs framework for Big Data?

(a) Jim Gray
(b) **Doug Laney** ← correct
(c) Jeffrey Dean
(d) Erik Brynjolfsson

Q2. Which of the following is an example of **unstructured data**?

(a) A SQL database table of student records
(b) A CSV file of monthly sales figures
(c) **An MRI scan image** ← correct
(d) A JSON array of product prices

Q3. The CAP theorem states that a distributed system cannot simultaneously guarantee more than two of which three properties?

(a) Cost, Availability, Performance
(b) **Consistency, Availability, Partition Tolerance** ← correct
(c) Correctness, Atomicity, Persistence
(d) Concurrency, Accuracy, Partitioning

Q4. Which "V" of Big Data refers to the **speed** at which data is generated and must be processed?

(a) Volume
(b) Variety
(c) **Velocity** ← correct
(d) Veracity

True / False

Instructions: State True or False and write a one-sentence justification.

Statement	Answer
1. Structured data accounts for approximately 80% of all digital data.	**False**
2. The 3Vs framework was later extended to 5Vs by adding Veracity and Value.	**True**
3. Horizontal scaling means adding more RAM or CPU to a single existing server.	**False**
4. Batch processing is suitable for real-time fraud detection in payment systems.	**False**

Justifications:

1. *False* — Structured data is only ∼20%; approximately 80% of all digital data is unstructured (images, video, free text).
2. *True* — Gartner's Beyer & Laney (2012) formally extended the 3Vs to 5Vs, adding Veracity (data quality uncertainty) and Value (business utility).
3. *False* — Horizontal scaling adds more machines to a cluster (scale out). Adding more hardware to a single server is vertical scaling (scale up).
4. *False* — Batch processing has latency of minutes to hours. Fraud detection requires sub-second decision-making, which requires stream processing.

Descriptive Questions

Instructions: Answer each question in 100–150 words.

D1. Explain the 3Vs (and optionally 5Vs) of Big Data with one real-world example for each dimension.

D2. Differentiate between structured, semi-structured, and unstructured data with examples drawn from the healthcare domain (hospital, clinic, or public health).

D3. What is the CAP theorem? State each of the three properties in one sentence each. Why is Partition Tolerance considered non-negotiable for large-scale distributed systems?

Analytical Questions

Instructions: Answer in 200–300 words with step-by-step reasoning.

A1. 5Vs Analysis of a Ride-Sharing Platform:
A ride-sharing company collects GPS pings every 3 seconds from 5 million active drivers. Each ping includes driver ID, latitude, longitude, speed, and battery level. Analyse this scenario using the full **5Vs** framework. For each V: (a) describe the specific challenge, and (b) propose a technical approach to address it. Which V poses the *greatest* engineering challenge for this system and why?

A2. Batch vs. Stream Processing Decision:
You are the lead data engineer for a large Bangladeshi bank. The bank needs to: (i) detect fraudulent transactions within 200 milliseconds, (ii) generate end-of-month customer account statements, (iii) retrain the fraud detection ML model weekly on the past 6 months of transactions, and (iv) power a live dashboard showing the total number of active sessions across all branches. For each of the four requirements, specify whether you would use batch processing, stream processing, or both, and justify your choice with reference to the latency, volume, and fault-tolerance properties discussed in this lecture.

Textbook References for Week 1

- **[SA]** Seema Acharya & Subhashini Chellappan. *Big Data and Analytics.* Wiley, 2015. **Chapter 1**
  Chapter 1 provides a foundational introduction to big data: the 3Vs, types of data, business drivers, and an overview of the Hadoop ecosystem. Primary reading for this week.

- **[TE]** Thomas Erl, Wajid Khattak & Paul Buhler. *Big Data Fundamentals.* Prentice Hall, 2016. **Chapter 1**
  Chapter 1 defines big data from an enterprise architecture perspective, extending the 3Vs to include operational and enterprise big data types.

- **[MMS]** Viktor Mayer-Schönberger & Kenneth Cukier. *Big Data: A Revolution.* Houghton Mifflin, 2013. **Chapter 1**
  Chapter 1 ("Now") provides a narrative account of how big data is transforming society — excellent context for the "why it matters" framing of this week.

- **[LRU]** Leskovec, Rajaraman & Ullman. *Mining of Massive Datasets*, 3rd ed. Cambridge Univ. Press, 2020. **Chapter 1**
  Chapter 1 frames the problem from a computational perspective: algorithms that must run on data too large for a single machine. Available free at `mmds.org`.

- **Seminal Paper:** Doug Laney (2001). *"3D Data Management: Controlling Data Volume, Velocity and Variety."* META Group Research Note, 6 Feb 2001. [The original 3Vs paper — 2 pages, recommended reading]

- **Seminal Paper:** Eric Brewer (2000). *"Towards Robust Distributed Systems."* Keynote, ACM PODC 2000. [Original CAP conjecture; formalized by Gilbert & Lynch, ACM SIGACT News, 2002]